

02 — LLM Setup via SAP Gen AI Hub (Deep Dive)

Module purpose: the standard way every higher-level template (query agent, RAG, chatbot) gets a callable, LangChain-compatible LLM or embedding model from SAP AI Core — without any provider-specific SDK code.

Chat Model Setup

```
import os
from dotenv import load_dotenv
from gen_ai_hub.proxy.langchain.openai import ChatOpenAI          # LangChain-compatible
↳ wrapper class
from gen_ai_hub.proxy.core.proxy_clients import get_proxy_client

load_dotenv()

def get_llm(model: str = "gpt-4o-mini", temperature: float = 0):
    deployment_id = os.getenv("LLM_DEPLOYMENT_ID")                # from AI Launchpad -> AI Core ->
    ↳ Deployments
    if not deployment_id:
        raise EnvironmentError("LLM_DEPLOYMENT_ID not set.")

    proxy_client = get_proxy_client("gen-ai-hub")                 # resolves AICORE_* auth automatically

    llm = ChatOpenAI(
        proxy_model_name = model,                                # logical model name understood by the proxy
        proxy_client      = proxy_client,
        deployment_id     = deployment_id,                       # WHICH provisioned instance actually serves calls
        temperature       = temperature,                         # 0 = deterministic, 1 = creative
    )
    return llm

# Usage — LangChain-native call:
from langchain.schema import HumanMessage
response = get_llm().invoke([HumanMessage(content="Say hello in one sentence.")])
print(response.content)
```

Working explanation: `get_proxy_client("gen-ai-hub")` is the piece doing the heavy lifting — it silently resolves your `AICORE_CLIENT_ID/SECRET/AUTH_URL/BASE_URL` credentials (from env vars, `~/.aicore/config.json`, or `VCAP_SERVICES`), fetches/refreshes an OAuth2 token, and returns a client object the `ChatOpenAI` wrapper uses internally on every `.invoke()` call. The returned `llm` object is a genuine LangChain `ChatOpenAI` instance — it works with `|` pipe chains, `ChatPromptTemplate`, agents, and any other LangChain construct unmodified, which is why every downstream template (SQL generator, RAG chain, chatbot) can treat “the LLM” as a black box regardless of which provider is actually behind the SAP AI Core deployment.

Embedding Model Setup (same pattern, different class + output shape)

```
from gen_ai_hub.proxy.langchain.openai import OpenAIEmbeddings

EMBEDDING_MODEL = "text-embedding-ada-002"
VECTOR_DIMENSION = 1536                                # MUST match the REAL_VECTOR(n) column dimension in HANA
↳ (sheet 04)
```

```
def get_embedding_model():
    deployment_id = os.getenv("EMBEDDING_DEPLOYMENT_ID")
    proxy_client = get_proxy_client("gen-ai-hub")
    model = OpenAIEmbeddings(
        proxy_model_name = EMBEDDING_MODEL,
        proxy_client      = proxy_client,
        deployment_id     = deployment_id,
    )
    return model

# Two entry points:
vectors = model.embed_documents(["text one", "text two"]) # -> list[list[float]], for
↳ storing many chunks
vector  = model.embed_query("a search question")         # -> list[float], for a single
↳ search query
```

Working explanation: `embed_documents` and `embed_query` exist as two separate methods because some embedding providers actually generate slightly different vectors for a “document” vs. a “query” (asymmetric embedding models) — using the right method for the right side of the operation keeps you compatible with any such model even though `text-embedding-ada-002` itself treats them identically. The **dimension must match exactly** what your HANA `REAL_VECTOR(n)` column was created with, or `TO_REAL_VECTOR()` inserts will fail — 1536 for `ada-002`, 3072 for `text-embedding-3-large`.

build_llm(config) — the chatbot-flavoured variant

The chatbot template (sheet 06) wraps this same pattern in a function that reads settings from a `ChatbotConfig` object instead of hardcoded defaults:

```
def build_llm(config: ChatbotConfig):
    proxy_client = get_proxy_client("gen-ai-hub")
    return ChatOpenAI(
        proxy_model_name=config.llm_model,
        proxy_client=proxy_client,
        deployment_id=config.llm_deployment_id,
        temperature=config.temperature,
        max_tokens=config.max_tokens,
    )

def build_embedding_model(config: ChatbotConfig):
    proxy_client = get_proxy_client("gen-ai-hub")
    return OpenAIEmbeddings(
        proxy_model_name=config.embedding_model,
        proxy_client=proxy_client,
        deployment_id=config.embedding_deployment_id,
    )
```

This is the same two functions, just config-driven instead of parameter-driven — the payoff is that `ChatbotConfig` (one dataclass) can now fully control model choice for the entire chatbot.

Key Points

- `deployment_id` $\neq$ model name: `deployment` = your provisioned, billable **instance** on AI Core; model name tells the proxy which underlying model that instance serves.
- `get_proxy_client("gen-ai-hub")` handles auth token acquisition/refresh — you never touch OAuth2 directly.
- Both `ChatOpenAI` and `OpenAIEmbeddings` are **real LangChain classes** — full ecosystem compatibility (chains, agents, output parsers) comes for free.
- Embedding dimension is a **hard contract** between the model you embed with and the HANA table schema you store into — mismatches are a very common bug.
- `temperature=0` is used everywhere SQL/JSON/deterministic output matters (query agent, structured output); higher temperature only where creative variation is desired.
- Two chat model deployments in the same project (`LLM_DEPLOYMENT_ID`) and one embedding deployment (`EMBEDDING_DEPLOYMENT_ID`) are typically configured separately in AI Launchpad — they are billed and scaled independently.

Interview Q&A

Q: What does “harmonized API” mean in the Gen AI Hub context, concretely? A: Every supported provider (OpenAI, Google, Anthropic, Amazon) is exposed through the same LangChain-compatible class shape — swapping `proxy_model_name` (and its deployment) is a config change, not a rewrite of calling code.

Q: Why are there separate `embed_documents` and `embed_query` methods instead of one `embed()`? A: Some embedding models are asymmetric — they compute different vector representations for “things to be searched” vs. “the search query itself” — having two methods keeps your code correct even if you later swap in a model where that distinction matters.

Q: What happens if you change `EMBEDDING_MODEL` from `ada-002` to `text-embedding-3-large` without updating anything else? A: Inserts into the existing `REAL_VECTOR(1536)` HANA column will fail or truncate/error because the new model produces 3072-dimension vectors — you’d need to also update `VECTOR_DIMENSION` and recreate/migrate the vector table (sheet 04).

Q: When would you use the proxy (this sheet) vs. the Orchestration Service (sheets 07-08)? A: Proxy for a single model call with no extra governance needs — simpler, lower latency. Orchestration when you need masking/filtering/grounding/translation applied consistently and centrally, without hand-writing that logic in every service that calls the LLM.